

Ejercicios 21-Oct-2025

Práctica – Peticiones GET con la API ReqRes

Objetivos del día

1. Obtener un único usuario o una lista de usuarios

A partir de la documentación oficial de la API  <https://reqres.in/>

Se debe implementar tanto la obtención de un solo registro (`/users/{id}`) como la lista completa (`/users`).

2. Globalizar variables y constantes en el código

Evitar repeticiones mediante constantes globales o variables compartidas:

- Base URL de la API
- Variables de entorno (API Key si existiera)
- Parámetros de consulta (`PATH` , `page` , etc.)

3. Manejo de errores con `try` / `catch` y verificación de códigos de estado (`statusCode`)

Analizar qué ocurre si la respuesta es `200 OK` o `404 Not Found` .

Mostrar mensajes de error o de éxito según el caso.

Ejercicio 1 – Listar y obtener usuarios

Endpoint:

- `GET https://reqres.in/api/users`
- `GET https://reqres.in/api/users/{id}`

Tareas:

1. Crear los métodos base:

```
Future<Persona?> getSingleUserObject(int id) async {  
    return null;  
}  
  
Future<List<Persona>> getListPersonasObject() async {  
    return [];  
}
```

2. Implementar las peticiones usando `http.get` .
3. Decodificar el JSON y mapearlo a la clase `Persona` .
4. Aplicar `try/catch` y mostrar mensajes según el `statusCode` .

Ejercicio 2 – Trabajar con el endpoint “resources”

Endpoint:

- `GET https://reqres.in/api/resource`

Objetivos:

1. **Crear modelos**

- Define una clase `Recurso` con los campos del JSON (`id` , `name` , `year` , `color` , `pantone_value`).

2. **Realizar las peticiones**

- Obtener todos los recursos (`/resource`).

3. **Mostrar datos**

- Imprimir por consola los valores formateados

4. **Manejo de errores**

- Verificar comportamiento ante códigos 200 y 404.

Actividad extra:

Relacionar el modelo `Recurso` con el de `Persona` y analizar similitudes entre ambos.

Ejercicio 3 – Modelo de respuesta general

La mayoría de endpoints de ReqRes comparten una estructura común:

```
{
  "page": 1,
  "per_page": 6,
  "total": 12,
  "total_pages": 2,
  "data": [ ... ]
}
```

Tareas:

1. **Crear un modelo genérico** `ApiResponse<T>` que permita reutilizar esta estructura con distintos tipos de datos (`Persona` , `Recurso` , etc.).
 2. **Analizar los datos de respuesta**
 - Diferenciar entre los campos de paginación y el contenido (`data`).
 3. **Integrar con los modelos existentes**
 - Usar `ApiResponse<Persona>` y `ApiResponse<Recurso>` para tipar correctamente las peticiones.
-

Recomendaciones técnicas

- Utiliza constantes globales:

```
const String baseUrl = "https://reqres.in/api";
```

- Centraliza las peticiones en una clase `ReqResService` .

- Añade manejo de errores:

```
if (res.statusCode == 404) {  
    print("Recurso no encontrado");  
}
```

- Prueba tus peticiones con IDs inexistentes para validar los errores.